

TRAVEL MEMORIES

مستندات فنی پروژه (Technical Documentation)

اپلیکیشن موبایل سفر و خاطرات سفر — ساخته شده با Flutter

Repository: github.com/zahrazn21/Travel_Memories

زهرآزنده بودی - ۴۰۱۰۷۱۱۳۱۶

۱. معرفی پروژه
۲. تکنولوژی‌ها و پکیج‌های اصلی
۳. ساختار پروژه (Project Structure)
۴. معماری کلی برنامه
۵. مدل‌های داده (Models)
۶. سرویس‌ها (Services)
۷. صفحات برنامه (Screens)
۸. ویجت‌های اختصاصی (Widgets)
۹. سیستم تم و ظاهر برنامه
۱۰. احراز هویت (Authentication)
۱۱. متغیرهای محیطی (Environment Variables)
۱۲. راه‌اندازی و اجرای پروژه
۱۳. پلتفرم‌های پشتیبانی شده

۱. معرفی پروژه

Travel Memories یک اپلیکیشن موبایل چندسکویی (Flutter) است که به کاربران کمک می‌کند شهرها و جاذبه‌های گردشگری ایران را کاوش کنند، اطلاعات آب‌وهوایی و تاریخی هر مکان را ببینند، مکان‌های موردعلاقه را ذخیره کنند و خاطرات سفر شخصی خود (شامل عکس، تاریخ و توضیحات) را ثبت و مدیریت نمایند.

برنامه به زبان فارسی طراحی شده (راست‌به‌چپ) و از تقویم شمسی، فونت‌های فارسی اختصاصی و API‌های عمومی مانند Wikidata، Wikipedia، Wikimedia Commons و Open-Meteo برای دریافت داده‌های واقعی جاذبه‌ها و آب‌وهوا استفاده می‌کند.

قابلیت‌های اصلی

- مرور شهرهای ایران به همراه اطلاعات جمعیت، مساحت و توضیحات از یک دیتاست محلی (data/cities.json)
- نمایش جاذبه‌های گردشگری هر شهر با اعلام زنده از Wikidata (SPARQL) و تصاویر از Wikipedia/ Commons
- نمایش آب‌وهوای لحظه‌ای و پیش‌بینی روزانه هر شهر (Open-Meteo API)
- نمایش موقعیت جغرافیایی شهرها و جاذبه‌ها روی نقشه (flutter_map / Google Maps)
- افزودن، ویرایش و مدیریت «خاطرات سفر» همراه با عکس و تاریخ شمسی
- سیستم موردعلاقه‌ها (Favorites) برای جاذبه‌ها و خاطرات، جداگانه از هم
- جست‌وجوی شهرها و جاذبه‌ها
- احراز هویت کاربر (ثبت‌نام / ورود) با ذخیره‌سازی محلی و رمزنگاری SHA-۲۵۶
- پروفایل کاربری و امکان تغییر تم بصری برنامه (سه تم: تیره، آبی، صورتی)

۲. تکنولوژی‌ها و پکیج‌های اصلی

این پروژه با Flutter (SDK ۳.۱۱.۰) نوشته شده است. جدول زیر مهم‌ترین وابستگی‌های تعریف‌شده در pubspec.yaml را نشان می‌دهد:

پکیج	کاربرد در پروژه
flutter / flutter_localizations	فریم‌ورک اصلی و پشتیبانی چندزبانه (فارسی/انگلیسی)
provider	مدیریت وضعیت (State Management) — برای مثال ThemeProvider
http / dio	درخواست‌های شبکه‌ای به API‌های خارجی
flutter_dotenv	بارگذاری متغیرهای محیطی از فایل env.
shared_preferences	ذخیره‌سازی محلی کلید-مقدار (کاربران، موردعلاقه‌ها، خاطرات)

پکیج	کاربرد در پروژه
hive / hive_flutter	دیتابیس محلی سبک (NoSQL) — اکنون فعالانه برای کش پایدار جاذبه‌های هر شهر استفاده می‌شود (به‌روزرسانی جدید)
cached_network_image	کش کردن تصاویر دانلود شده از اینترنت روی دیسک برای بارگذاری سریع‌تر (به‌روزرسانی جدید)
flutter_cache_manager	مدیریت کش فایل‌ها، پیش‌نیاز cached_network_image (به‌روزرسانی جدید)
flutter_launcher_icons	تولید خودکار آیکن اپلیکیشن برای اندروید/iOS از یک تصویر منبع (به‌روزرسانی جدید)
path_provider	دسترسی به مسیرهای فایل سیستم دستگاه
image_picker	انتخاب عکس از گالری یا دوربین برای خاطرات سفر
flutter_map / latlong	نمایش نقشه متن‌باز (OpenStreetMap) برای شهرها و جاذبه‌ها
google_maps_flutter	پشتیبانی نقشه گوگل (به‌صورت جایگزین/تکمیلی)
shamsi_date / persian_datetime_picker	تقویم و انتخاب‌گر تاریخ شمسی
url_launcher	باز کردن لینک‌های خارجی (مثلاً وب‌سایت جاذبه)
crypto	هش کردن رمز عبور کاربران (SHA-۲۵۶)
cupertino_icons	آیکن‌های سبک iOS

APIهای خارجی مورد استفاده

- Wikidata Query Service (SPARQL) — دریافت لیست جاذبه‌های نزدیک هر شهر
- Wikipedia API (fa / en) — دریافت تصویر بندانگشتی جاذبه‌ها
- Wikimedia Commons API — منبع جایگزین برای تصاویر
- Open-Meteo API — دریافت دمای فعلی و پیش‌بینی روزانه بدون نیاز به کلید API

۳. ساختار پروژه (Project Structure)

این پروژه از ساختار استاندارد Flutter پیروی می‌کند و کد اصلی در پوشه lib/ سازمان‌دهی شده است:

```

Travel_Memories/
├── lib/
│   ├── main.dart           # MaterialApp نقطه ورود برنامه و پیکربندی
│   ├── models/             # مدل‌های داده
│   │   ├── attraction.dart  # مدل جاذبه گردشگری
│   │   └── city.dart        # مدل شهر
│   ├── providers/          # مدیریت وضعیت تم برنامه
│   │   └── theme_provider.dart
│   └── services/           # API منطق دسترسی به داده و

```

```

├── attractions_service.dart
├── auth_service.dart
├── city_data_service.dart
├── favorites_service.dart
├── memory_favorites_service.dart
├── weather_service.dart
├── weather_style.dart
├── wiki_service.dart
├── screens/                # برنامه UI صفحات
├── themes/                 # تعریف تمها و ظاهر برنامه
├── utils/                  # توابع کمکی
├── widgets/                # ویجت‌های قابل استفاده مجدد
│   └── retryable_network_image.dart # خودکار (جدید) retry تصویر شبکه با
├── data/
│   └── cities.json         # دیتاست محلی شهرهای ایران
├── fonts/                  # فونت‌های فارسی/انگلیسی اختصاصی
├── images/                 # تصاویر پس‌زمینه و آیکن‌ها
├── android/ ios/ web/ windows/ linux/ macos/ # کدهای مخصوص هر پلتفرم
├── test/
│   └── widget_test.dart
├── pubspec.yaml            # تعریف وابستگی‌ها، فونت‌ها و
└── README.md

```

۴. معماری کلی برنامه

برنامه از یک معماری لایه‌ای ساده پیروی می‌کند که در سه لایه اصلی قابل تفکیک است:

لایه UI (Screens & Widgets)

صفحات (lib/screens) مسئول نمایش رابط کاربری و مدیریت تعامل کاربر هستند. ویجت‌های اختصاصی (lib/widgets) عناصر قابل استفاده مجدد مثل کارت جاذبه، کارت آب‌وهوا و نوار پیمایش پایین را فراهم می‌کنند.

لایه سرویس (Services)

سرویس‌ها (lib/services) منطق دسترسی به داده — چه از منابع خارجی (Wikidata، Wikipedia، Open-Meteo) و چه از حافظه محلی (SharedPreferences) — را پیاده‌سازی می‌کنند. اغلب سرویس‌ها به صورت Singleton طراحی شده‌اند (مثل FavoritesService.instance) و برخی از آن‌ها با ChangeNotifier ترکیب شده‌اند تا تغییرات را به UI اطلاع دهند.

لایه مدل (Models)

مدل‌های داده (lib/models) ساختار داده‌های اصلی برنامه، یعنی City و Attraction، را با متدهای تبدیل به/از JSON یا Map تعریف می‌کنند.

مدیریت وضعیت (State Management)

برنامه از پکیج provider برای مدیریت وضعیت سراسری (به‌ویژه تم برنامه از طریق ThemeProvider) استفاده می‌کند. برخی سرویس‌ها نیز خودشان ChangeNotifier هستند و مستقیماً به عنوان singleton در ویجت‌ها استفاده می‌شوند (FavoritesService، MemoryFavoritesService).

جریان راه اندازی برنامه (main.dart)

- مقداردهی اولیه Flutter binding
 - بارگذاری موردعلاقه‌های ذخیره شده (FavoritesService.instance.load)
 - بارگذاری فایل env با flutter_dotenv
 - بررسی وضعیت ورود کاربر (AuthService.instance.isLoggedIn)
 - اجرای برنامه با ThemeProvider<ChangeNotifierProvider> و نمایش LoginScreen یا MainScreen بسته به وضعیت ورود
- MainScreen یک IndexedStack شامل پنج تب اصلی است: جست‌وجو، خاطرات، خانه، موردعلاقه‌ها و پروفایل که با CustomBottomNav جابه‌جا می‌شوند.

۵. مدل‌های داده (Models)

City

مدل شهر، اطلاعات پایه هر شهر ایرانی را نگه می‌دارد و از فایل data/cities.json ساخته می‌شود:

```
class City {
  final int id;
  final String nameFa;      // نام فارسی شهر
  final String nameEn;      // نام انگلیسی شهر
  final String provinceFa;  // نام استان
  final double lat, lng;    // مختصات جغرافیایی
  final int population;     // جمعیت
  final String wikiEn;      // عنوان صفحه ویکی‌پدیا
}
```

Attraction

مدل جاذبه گردشگری، اطلاعات دریافت‌شده از Wikidata را نگه می‌دارد و متدهای fromMap و toMap برای سریالایز کردن (مثلاً ذخیره در موردعلاقه‌ها یا کش Hive) دارد:

```
class Attraction {
  final String name;
  final String? description;
  final String? website;
  final String? inceptionYear; // سال ساخت/ثبت
  String? image;              // آدرس تصویر (جدید: قابل تغییر است)
  final double? lat, lng;
  final String? province;
  final String? nameEn;       // نام انگلیسی (فیلد جدید)
}
```

توجه: عملگر == و hashCode روی Attraction بر اساس فیلد name بازنویسی شده‌اند؛ یعنی دو جاذبه با نام یکسان از نظر برنامه یکی در نظر گرفته می‌شوند.

تغییرات این نسخه: فیلد image دیگر final نیست و می‌تواند بعداً (مثلاً وقتی کاربر یا سرویس تصویر بهتری پیدا می‌کند) به‌روزرسانی شود؛ همچنین فیلد جدید nameEn برای نگه‌داشتن نام انگلیسی جاذبه اضافه شده که در جست‌وجوی تصویر از ویکی‌پدیا/Commons استفاده می‌شود.

۶. سرویس‌ها (Services)

سرویس‌ها منطق اصلی دسترسی به داده در برنامه را تشکیل می‌دهند. جدول زیر خلاصه‌ای از مسئولیت هر سرویس را نشان می‌دهد:

سرویس	مسئولیت
AttractionsService	استعلام SPARQL از Wikidata برای یافتن جاذبه‌های نزدیک یک مختصات جغرافیایی؛ دریافت تصویر از Wikipedia/Commons؛ دارای cache حافظه‌ای بر اساس cityKey.
AuthService	ثبت‌نام، ورود و خروج کاربر با ذخیره در SharedPreferences؛ هش رمز عبور با SHA-۲۵۶؛ بررسی وضعیت ورود.
CityDataService	بارگذاری و جست‌وجوی شهرها از فایل محلی data/cities.json.
FavoritesService	مدیریت لیست جاذبه‌های موردعلاقه (افزودن/حذف/toggle) با پایبندی در SharedPreferences و اطلاع‌رسانی از طریق ChangeNotifier.
MemoryFavoritesService	مشابه FavoritesService اما برای خاطرات سفر؛ شامل انکود/دیکود تصاویر Base۶۴ و تاریخ‌های DateTime.
WeatherService	دریافت دمای فعلی و پیش‌بینی حداقل/حداکثر روزانه از Open-Meteo برای یک مختصات مشخص.
WeatherStyle	تبدیل کد آب‌وهوای Open-Meteo به آیکن، رنگ و برجسب فارسی مناسب (آفتابی، بارانی، برفی و ...).
WikiService	جست‌وجوی تصویر از Wikipedia و Wikimedia Commons با منطق fallback بین فارسی/انگلیسی؛ دارای cache.

جزئیات AttractionsService (به‌روزرسانی شده)

این سرویس با ارسال یک کوئری SPARQL به query.wikidata.org، جاذبه‌های گردشگری (اماکن تاریخی، فرهنگی و طبیعی) را در شعاع ۴۰ کیلومتری یک مختصات جستجو می‌کند و تا ۲۰۰ نتیجه (قبلاً ۸۰ نتیجه) دریافت می‌کند. کوئری، انواعی مانند موزه، قلعه، بازار، دریاچه، آبشار، کوه و غار را شامل شهر/روستا نمی‌کند (با MINUS فیلتر شده‌اند) و نتایج را بر اساس تعداد sitelinks (میزان شناخته‌شدگی) مرتب می‌کند.

نسخه جدید کوثری، مستقیماً تصویر رسمی جاذبه (ویژگی P۱۸ در Wikidata) و دسته Wikimedia Commons مرتبط (ویژگی P۳۷۳) را نیز از Wikidata درخواست می‌کند؛ یعنی برای بسیاری از جاذبه‌ها دیگر نیازی به جست‌وجوی جداگانه تصویر نیست.

کش دو لایه: حافظه + Hive

علاوه بر کش حافظه‌ای موقت قبلی، اکنون یک کش پایدار مبتنی بر Hive (جعبه attractions_cache_box) اضافه شده که نتایج هر شهر را به همراه زمان ذخیره‌سازی نگه می‌دارد و تا ۷ روز (cacheTtl_) معتبر است. با فراخوانی دوباره fetchNear برای یک شهر، ابتدا کش حافظه، سپس کش Hive (در صورت تازه بودن) و در نهایت درخواست تازه از Wikidata بررسی می‌شود. پارامتر forceRefresh امکان نادیده گرفتن کش و دریافت مجدد را می‌دهد؛ متد clearCache(cityKey) نیز کش حافظه و Hive یک شهر را پاک می‌کند.

استراتژی چندمرحله‌ای یافتن تصویر

برای جاذبه‌هایی که تصویر مستقیم از Wikidata ندارند، سرویس به صورت مرحله‌ای (حداکثر ۵ دور، هر دور با batch یک تا دو موردی) اقدام به جست‌وجوی تصویر می‌کند: ابتدا از دسته Wikimedia Commons مرتبط (P۳۷۳)، سپس ویکی‌پدیای انگلیسی، سپس ویکی‌پدیای فارسی، سپس جست‌وجوی آزاد در Commons با نام انگلیسی/فارسی، و در نهایت با عبارت‌های ترکیبی مانند «نام + جاذبه» یا «name + landmark». نتایج نامعتبر (فایل‌های svg/pdf/tiff، پرچم ایران، نقشه خالی و مانند آن) با متد isValidImage فیلتر می‌شوند و آدرس نهایی با normalizeImageUrl اصلاح می‌شود (تبدیل http به https و افزودن پارامتر عرض تصویر).

پارامتر onBatchUpdate در fetchNear امکان می‌دهد UI به صورت تدریجی (progressive) به محض پیدا شدن هر گروه تصویر جدید به روزرسانی شود، بدون آنکه منتظر پایان کامل فرایند بماند.

مقاوم‌سازی درخواست‌های شبکه

متد داخلی getWithRetry_ با منطق backoff، خطای ۴۲۹ (Rate Limit) و خطاهای سرور ۵xx را با تلاش مجدد (حداکثر ۳ بار) مدیریت می‌کند؛ همچنین یک هدر User-Agent اختصاصی به تمام درخواست‌های Wikipedia/Commons/Wikidata اضافه شده است. پردازش JSON و حذف موارد تکراری (dedup) نیز با compute() در یک ایزوله جداگانه اجرا می‌شود تا از فریز شدن UI جلوگیری شود.

متد updateAttractionImage

این متد جدید امکان می‌دهد وقتی کاربر یا صفحه جزئیات جاذبه، تصویر بهتری برای یک جاذبه پیدا می‌کند (مثلاً از گالری تصاویر ویکی‌پدیا)، آن تصویر هم در کش حافظه و هم در کش پایدار Hive به روزرسانی و ذخیره شود؛ به این ترتیب دفعات بعدی که کاربر آن شهر را باز می‌کند، تصویر بهتر از کش خوانده می‌شود.

در پایان فرایند، جاذبه‌هایی که نه تصویر و نه توضیحی دارند، از فهرست نهایی حذف می‌شوند تا کیفیت داده نمایش داده شده در UI بالاتر برود.

جزئیات AuthService

این سرویس یک سیستم احراز هویت کاملاً محلی (بدون بک‌اند) پیاده‌سازی می‌کند: کاربران در قالب یک Map در SharedPreferences (کلید users) ذخیره می‌شوند و رمز عبور آن‌ها با SHA-۲۵۶ هش می‌شود. ایمیل کاربر وارد شده هنگام ورود یا ثبت‌نام نرمال‌سازی (trim + lowercase) می‌شود.

۷. صفحات برنامه (Screens)

جدول زیر صفحات اصلی برنامه و فایل مربوط به هر کدام را نشان می‌دهد:

صفحه	فایل	توضیح
HomeScreen	home_screen.dart	صفحه اصلی؛ نمایش شهرهای پیشنهادی، کارت آب‌وهوا و دسترسی به همه شهرها/جاذبه‌ها.
SearchScreen	search_screen.dart	جست‌وجوی شهرها و جاذبه‌ها.
AllCitiesScreen	all_cities_screen.dart	نمایش فهرست کامل شهرها.
AttractionsScreen	attractions_screen.dart	نمایش جاذبه‌های یک شهر خاص به همراه آب‌وهوا و نقشه.
AllAttractionsScreen	all_attractions_screen.dart	نمایش کامل جاذبه‌های یک شهر.
AttractionDetailScreen	attraction_detail_screen.dart	جزئیات یک جاذبه؛ امکان افزودن خاطره مرتبط، گالری تصاویر با اعتبارسنجی/نرمال‌سازی خودکار آدرس عکس‌ها، دکمه موردعلاقه واقعاً متصل به FavoritesService، نقشه کوچک محل جاذبه (flutter_map) و باز کردن وب‌سایت جاذبه با url_launcher. (به‌روزرسانی جدید)
CityMapWidget / FullMapScreen	city_map_screen.dart, full_map_screen.dart	نمایش نقشه کوچک و نقشه تمام‌صفحه شهر با flutter_map؛ اکنون دکمه «باز کردن در اپ نقشه» نیز اضافه شده که با url_launcher مسیر را در Google Maps، Apple Maps یا نسخه وب باز می‌کند. (به‌روزرسانی جدید)
MemoriesListPage	memories_screen.dart	فهرست خاطرات ثبت‌شده کاربر.
AddMemoryScreen	add_memory_screen.dart	فرم افزودن/ویرایش خاطره سفر شامل عکس، عنوان، تاریخ شمسی و توضیحات.
FavoritesScreen	favorites_screen.dart	نمایش جاذبه‌ها و خاطرات موردعلاقه.
LoginScreen / RegisterScreen	login_screen.dart, register_screen.dart	فرم‌های ورود و ثبت‌نام کاربر.
ProfileScreen	profile_screen.dart	پروفایل کاربر، خروج از حساب و تغییر تم برنامه.

پیمایش بین پنج تب اصلی (جست‌وجو، خاطرات، خانه، موردعلاقه‌ها، پروفایل) از طریق ویجت CustomBottomNav در main.dart مدیریت می‌شود. توجه شود که فایل cities_detail_screen.dart در مخزن خالی است و به نظر می‌رسد در حال حاضر استفاده نمی‌شود.

۸. ویجت‌های اختصاصی (Widgets)

ویجت‌های زیر در lib/widgets تعریف شده و در چند صفحه به صورت مشترک استفاده می‌شوند:

ویجت	کاربرد
AttractionCard	کارت نمایش خلاصه یک جاذبه (تصویر، نام، توضیح کوتاه)
AttractionCardSkeleton	حالت اسکلتی (Loading Skeleton) برای کارت جاذبه هنگام بارگذاری
CityStatItem	نمایش یک آمار شهر (مثلاً جمعیت یا مساحت)
CityWeatherCard	کارت نمایش آب‌وهوای شهر
CustomBottomNav	نوار پیمایش پایین صفحه با پنج آیتم
SeasonalPickCard	کارت پیشنهاد فصلی مقاصد سفر
ShimmerBox	افکت shimmer برای حالت بارگذاری محتوا
WaveClipper	برش موجی سفارشی برای پس‌زمینه بخش‌های UI
WeatherCard	کارت نمایش اطلاعات آب‌وهوا
RetryableNetworkImage (جدید)	جایگزین Image.network با قابلیت کش دیسک (cached_network_image)، تلاش خودکار مجدد در صورت خطا، و امکان تعیین یک fallbackBuilder سفارشی؛ در AttractionCard جایگزین Image.network قبلی شده است.

جزئیات RetryableNetworkImage (جدید)

این ویجت با استفاده از پکیج cached_network_image، تصویر را ابتدا از کش دیسک بارگذاری می‌کند و در صورت نبود، از شبکه دانلود و کش می‌کند. در صورت بروز خطا، به صورت خودکار تا maxAutoRetries بار (پیش فرض ۲ بار) با تأخیر افزایشی دوباره تلاش می‌کند؛ اگر باز هم ناموفق بود، یا از fallbackBuilder اختصاصی استفاده می‌شود یا یک آیکون «تلاش دوباره» قابل کلیک نمایش داده می‌شود تا کاربر بتواند دستی درخواست را تکرار کند.

به روزرسانی AttractionCard

کارت جاذبه اکنون از RetryableNetworkImage به جای Image.network استفاده می‌کند و در صورت نبود تصویر یا خطای مکرر، به تصویر محلی همان شهر (.../images/cities) به عنوان جایگزین سقوط می‌کند. همچنین اکنون نام شهر/استان زیر نام جاذبه نمایش داده می‌شود و کارت یک callback به نام onImageFound دارد که وقتی صفحه جزئیات جاذبه تصویر بهتری پیدا کند، آن را دریافت کرده و از طریق AttractionsService.updateAttractionImage در کش ذخیره می‌کند.

۹. سیستم تم و ظاهر برنامه

برنامه از سه تم بصری آماده پشتیبانی می‌کند که در `lib/themes/app_themes.dart` تعریف شده‌اند و از طریق `ThemeProvider` (پکیج `provider`) در سراسر برنامه در دسترس هستند:

- dark — تم تیره
- blue — تم آبی
- pink — تم صورتی (تم پیش‌فرض هنگام راه‌اندازی)

کاربر می‌تواند تم را از صفحه پروفایل تغییر دهد؛ این تغییر بلافاصله با `notifyListeners()` به کل برنامه اعمال می‌شود. علاوه بر تم رنگی، فایل `app_background_theme.dart` پس‌زمینه‌های گرافیکی صفحات و `app_menu_theme.dart` ظاهر منوها را تعریف می‌کند.

فونت و زبان

زبان پیش‌فرض برنامه فارسی (`fa_IR`) است و از `localizationsDelegate` های `PersianMaterialLocalizations` و `PersianCupertinoLocalizations` استفاده می‌شود تا عناصر بومی (مثل تقویم و اعداد) به‌درستی فارسی نمایش داده شوند. فونت اصلی متن `PlaypenSansArabic` است؛ علاوه بر آن فونت‌های `BalooBhaijaan۲`، `B-Nazanin`، `NotoNaskhArabic` و چند فونت لاتین دیگر نیز در `pubspec.yaml` ثبت شده‌اند.

۱۰. احراز هویت (Authentication)

سیستم احراز هویت برنامه کاملاً سمت کلاینت و بدون سرور پیاده‌سازی شده است:

- کاربران در `SharedPreferences`، به‌صورت یک `Map` با کلید ایمیل نرمال‌شده ذخیره می‌شوند.
 - رمز عبور هرگز به‌صورت متن ساده ذخیره نمی‌شود؛ با الگوریتم `SHA-۲۵۶` هش می‌شود.
 - پس از ورود یا ثبت‌نام موفق، ایمیل کاربر تحت کلید `logged_in_user` ذخیره و برای بررسی وضعیت ورود در دفعات بعدی استفاده می‌شود.
 - در صورت عدم ورود، کاربر به `LoginScreen` و در غیر این صورت به `MainScreen` هدایت می‌شود (تصمیم‌گیری در `main.dart`).
- توجه مهم: این روش صرفاً برای اهداف نمونه/آموزشی مناسب است و معادل یک سیستم احراز هویت امن مبتنی بر بک‌اند و توکن نیست؛ داده‌های کاربران فقط روی دستگاه محلی ذخیره می‌شوند و بین دستگاه‌ها همگام‌سازی نمی‌شوند.

۱۱. متغیرهای محیطی (Environment Variables)

برنامه از پکیج flutter_dotenv برای بارگذاری تنظیمات حساس از یک فایل .env در ریشه پروژه استفاده می‌کند. این فایل باید قبل از اجرای برنامه ساخته شود (در pubspec.yaml نیز به عنوان asset معرفی شده است) و در main.dart با دستور

```
dotenv.load(fileName: ".env")
```

بارگذاری می‌شود.

متغیرهای شناسایی شده در کد (lib/screens/home_screen.dart):

متغیر	کاربرد
API_KEY	کلید API مورد استفاده در HomeScreen برای فراخوانی سرویس بیرونی
BASE_URL	آدرس پایه سرویس بیرونی مورد استفاده در HomeScreen

نمونه فایل .env:

```
API_KEY=your_api_key_here
BASE_URL=https://example.com/api
```

توجه: فایل .env نباید در مخزن عمومی commit شود؛ پیشنهاد می‌شود در .gitignore قرار گیرد.

۱۲. راه‌اندازی و اجرای پروژه

پیش‌نیازها

- نصب Flutter SDK (سازگار با محدودیت $3.11.0^{\wedge}$ تعریف شده در pubspec.yaml)
- نصب Git
- یک شبیه‌ساز/دستگاه اندروید یا iOS، یا مرورگر برای اجرای نسخه وب

مراحل نصب

```
# ۱. کلون کردن مخزن
git clone https://github.com/zahrazn21/Travel_Memories.git
cd Travel_Memories

# ۲. ساخت فایل .env و افزودن مقادیر API_KEY و BASE_URL در ریشه پروژه و افزودن مقادیر .env

# ۳. نصب وابستگی‌ها
flutter pub get

# ۴. اجرای برنامه (روی دستگاه/شبیه‌ساز متصل)
flutter run

# اجرای نسخه وب
flutter run -d chrome
```

```
# اندروید release ساخت نسخه  
flutter build apk --release
```

تولید آیکن اپلیکیشن

پروژه اکنون از پکیج flutter_launcher_icons برای تولید خودکار آیکن اپ (از تصویر images/iconApp.png) برای اندروید و iOS استفاده می‌کند. برای بازتولید آیکن‌ها پس از تغییر تصویر منبع:

```
flutter pub get  
dart run flutter_launcher_icons
```

شناسه بسته (Application ID) اندروید نیز از com.example.myapp به com.example.travel_memories تغییر کرده و نام برنامه (android:label) به travel_memories به‌روزرسانی شده است.

۱۳. پلتفرم‌های پشتیبانی‌شده

این پروژه توسط ابزار flutter create با ساختار استاندارد چندپلتفرمی ساخته شده و پوشه‌های مربوط به هر پلتفرم را شامل می‌شود:

پلتفرم	پوشه
Android	/android
iOS	/ios
وب (Web)	/web
Windows	/windows
Linux	/linux
macOS	/macos